# Industrial Mathematical Models via the paceval MCP Server

Verified multi-million-character execution graphs, reusable computation handles and bulk evaluation for precision manufacturing

paceval | Scientific research report and industrial pilot proposal

13 November 2026

## Executive summary

Precision manufacturing and industrial digital twins require repeatable evaluation of mathematical models that depend on machine position, temperature, operating conditions and changing calibration parameters. An important research question is whether a **mathematical engine accessed directly through the Model Context Protocol (MCP)** can be used as a reusable computation service for such workloads. Unlike previous Python-based concept demonstrations, this study performs actual evaluations through the **connected paceval MCP server**. No Python client, local HTTP-compatible substitute, or local paceval library was used for the results reported here.

The main demonstrator is an intentionally synthetic four-variable thermal-compensation function constructed from 16,384 Gaussian radial-basis components. The complete expression has **2,359,315 characters**. The connected MCP service, identifying its mathematical runtime as **paceval 4.26**, created the complete graph successfully and evaluated it for twelve sets of input data by reusing the returned computation handle. A second experiment registered **eight independent 2,048-component graphs** and evaluated all eight graphs for twelve input sets in **one multi-computation batch request**. The modular outputs were aggregated in the documented graph-major order.

An independent JavaScript implementation evaluated the same synthetic mathematical expressions from the exact serialized decimal coefficients. The largest absolute discrepancy in the twelve-point monolithic comparison was approximately $\mathbf{5.3 \times 10^{-15}}$; the eight-graph experiment agreed to approximately $\mathbf{1.9 \times 10^{-15}}$. These figures confirm consistency for the tested inputs, not mathematical proof of correctness for all inputs or validity of a physical compensation model.

The industrial opportunity is **centralized, versioned, reusable mathematical execution** across engineering workflows: large formulas are registered once, then reused by their handles; batches allow many measurement records to be evaluated without retransmitting the full formula. Its likely economic benefit is simpler maintenance, model consistency and integration, rather than an assumed improvement in the computational speed of a specialized numerical kernel. The reported server timings are individual observations, not a statistically characterized end-to-end benchmark.

*Figure 1. Tested service concept. The mathematical execution was actually performed by the connected MCP tools. The industrial fleet workflow remains a proposed application; no factory integration is claimed.*

### Research outcomes at a glance

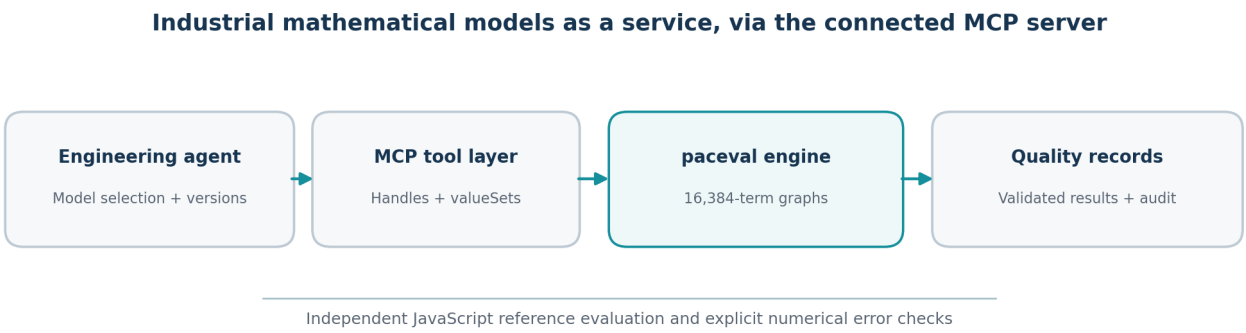



Figure 1: Research architecture: agent orchestration, MCP, paceval execution and independent verification

| Objective | Verified outcome |
|---|---|
| Extremely long server-side formula | 16,384 components; **2,359,315 characters** created and evaluated |
| Reusable graph handle | Yes, one handle was reused for multiple batches |
| Single-graph batch evaluation | **12 data records** for the largest expression |
| Multi-graph batch evaluation | **8 graphs × 12 records = 96 graph outputs** in one MCP call |
| Numerical cross-check | Independent JavaScript; errors on the order of $10^{-15}$ |
| Industrial accuracy and business benefit | **Not evaluated**; requires calibrated production measurements |

# 1. Industrial problem and scientific positioning

Thermal deformation, machine geometry and changes in the operating environment can create significant errors in precision machining and measurement. Established digital-twin research uses explicit models and measured data to estimate compensation fields. For example, published work has investigated integrated volumetric geometric and thermal correction, virtual sensor selection and uncertainty-aware compensation. The present project does **not** claim to invent thermal correction or Gaussian radial-basis models. Its research contribution is the verified **use of a standards-based MCP tool interface to create and execute extremely long mathematical expressions remotely, including both one-graph and multi-graph batch evaluation**. [1–4]

A suitable initial industrial application is supervisory analysis of a fleet of coordinate-measuring machines or machining centers. An engineering team may need to evaluate changing correction expressions for several machine configurations, repeat old calculations after a model revision, and produce auditable quality summaries. These operations often have looser latency requirements than servo control and can tolerate asynchronous communication with a central service.

The initial deployment should cover **non-actuating functions** only: offline replay, compensation recommendations, engineering what-if studies, quality report preparation and comparison of approved model versions. Production safety interlocks and hard real-time motion control must remain local and must not depend on an external MCP connection.

The resulting technical hypothesis is falsifiable: a centralized reusable mathematical component

provides measurable integration or maintenance value *if* mathematical equivalence, security, service reliability, lifecycle behavior and operating costs satisfy requirements defined with an industrial partner. A synthetic formula of great length is **not itself evidence of improved manufacturing accuracy**.

## 2. Existing paceval MCP interface and its verified capabilities

The paceval website announced the MCP server in August 2026 and documents the endpoint `https://mcp.paceval-service.com/mcp`. The public description includes reusable computation objects, deterministic mathematical execution and batch evaluation. In this study, the connected service exposed the following callable operations: [5]

| MCP operation | Use in this research |
|---|---|
| `paceval_create_computation` | Create a prepared expression from its text and variable order; obtain an opaque handle |
| `paceval_evaluate_many` | Evaluate one existing graph for multiple input vectors |
| `paceval_evaluate_multiple_many` | Evaluate multiple existing graphs for multiple input vectors in one bulk call |
| `paceval_get_error_information` | Inspect the error state of a previously created computation |
| `paceval_evaluate` and `paceval_evaluate_multiple` | Available operations for single-data-set experiments; not required for the main bulk comparison |

The service returned **paceval version 4.26** in the experimental responses. Each created handle was preserved as an opaque string and reused without resending the mathematical expression. The MCP result for the primary creation included the exact expression length, an error number of zero and a server-reported graph creation duration.

**Provenance limitation.** The connected MCP tool is genuinely available in this conversation and executed the mathematics. Its precise underlying network route, transport authentication and resource isolation cannot be independently inferred from the returned mathematical responses. The public endpoint is documented by paceval, but this experiment did not separately inspect its TLS/session metadata. The optional standalone JavaScript SDK client included with the publication package is reproducibility scaffolding; it was not executed across the network in this packaging environment. That distinction matters when reproducing the experiment outside the connected ChatGPT session.

## 3. Concrete research use case: analytical thermal compensation

Let the normalized geometric position be $(x, y, z) \in [0, 1]^3$ and the measured temperature be $T$ in degrees Celsius. Define a synthetic correction field

$$\delta_K(x, y, z, T) = 0.13 + 0.018(T - 20) + \sum_{k=1}^{K} a_k \exp\left[-\frac{(x-\mu_{kx})^2 + (y-\mu_{ky})^2 + (z-\mu_{kz})^2}{s_k}\right]\left[1 + c_k \frac{T-2}{20}\right.$$

The parameters $a_k$, $\mu_{k\cdot}$, $s_k$ and $c_k$ are deterministic **synthetic constants** generated from the component index $k$. Their exact generation rules and decimal serialization are given in `src/model.mjs`;

complete expressions and SHA-256 checksums are included in `generated/`. Numerical outputs are not assigned calibrated physical units because this expression has not been fitted to a machine.

The Gaussian basis is not the only plausible industrial model. Subsequent pilot extensions could combine thermal exponentials, analytical correction polynomials, segmented operating regions, environmental dependencies and nonlinear quality scores into the same reuse model. For a scientific comparison, however, the first experiment must use **one precisely specified function** in every execution path. Complexity should be raised systematically rather than assumed to be beneficial.

A supplementary quality indicator can be modeled by

$$q(x, y, z, T) = \left(\frac{T - 20}{30}\right)^2 + 0.2(x - y)^2 + 0.07(z - 0.5)^2.$$

This expression illustrates how one engineering workflow may combine several functions and score components. It is not a statistical probability or an approved machine-quality threshold.

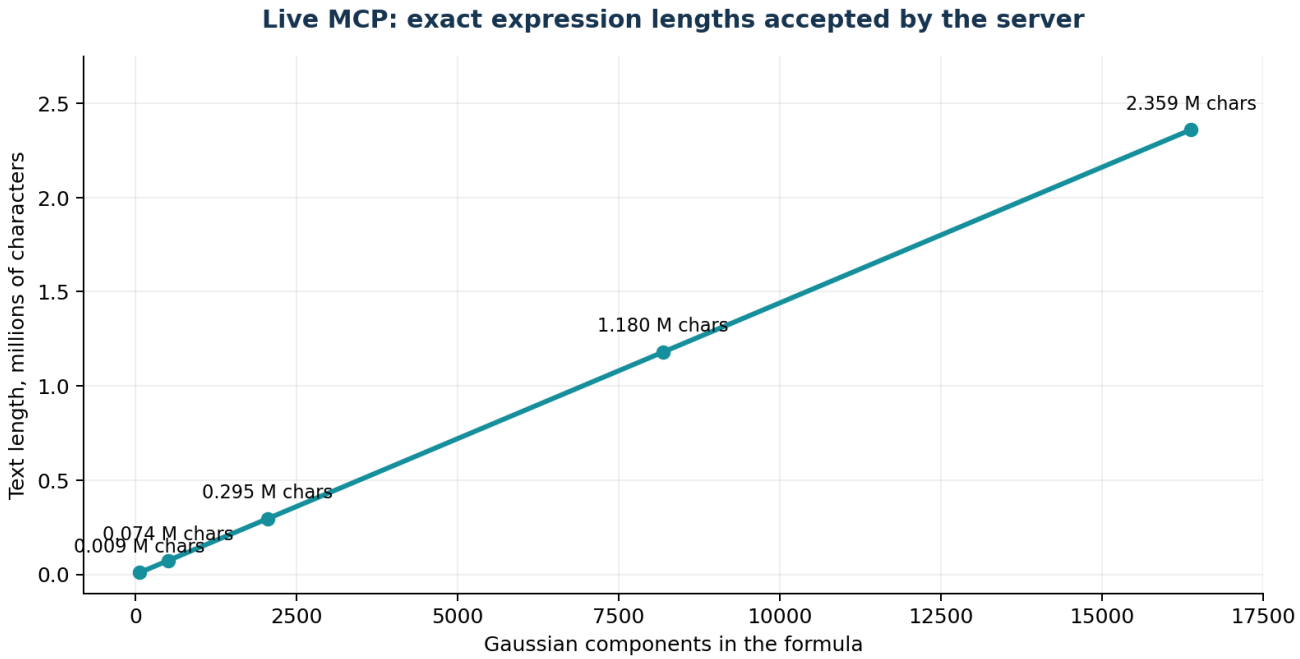



Figure 2: Exact length of the expressions accepted by the connected MCP server

*Figure 2. Exact lengths of the strings that the MCP server successfully parsed and evaluated. "Very long" here refers to expression text length, not any guaranteed upper bound of the service.*

# 4. Two genuinely tested execution architectures

## 4.1 One prepared, very long graph

The single-graph approach registers $\delta_{16384}$ once and retains its handle. New sets of inputs $\xi_j = (x_j, y_j, z_j, T_j)$ are then supplied to `paceval_evaluate_many`. The formula does not need to be retransmitted for every numerical evaluation. The service created the **2,359,315-character graph** and returned a successful result with no reported computational error. It also evaluated four individual stress points and then twelve input records using the same stored graph.

The server-reported graph creation time for this particular operation was **1.078361 seconds**, and one twelve-record batch was reported as **0.524339 seconds**. A repeat of that twelve-record calculation returned **0.512927 seconds**. The first three outputs were approximately 0.0166775468876766, 0.0340837875217513 and 0.119977647447063.

### 4.2 Eight prepared graphs in one multi-graph batch

To study modularity, the identical function was partitioned into eight additive components:

$$\delta_{16384} = \delta_1 + \delta_2 + \cdots + \delta_8,$$

where each $\delta_g$ contains 2,048 consecutive Gaussian terms and only the first graph includes $0.13 + 0.018(T - 20)$. The overall expression is mathematically the same. All graphs declare identical variable names in the same order: `x, y, z, temp`.

Eight graph-creation calls succeeded. The client then called `paceval_evaluate_multiple_many` **once**, passing the eight previously returned handles and twelve input vectors. The response contained **96 numerical values** and no nonzero error numbers. The response follows **graph-major ordering**: all twelve values belonging to graph 1, then all twelve for graph 2, and so forth. Results were combined per input using an independent JavaScript aggregation.

In this experiment, the server-reported time for the 96-result multi-graph batch was **0.767869 seconds**. The total mathematical text of the eight formulas was 2,359,308 characters; the seven-character difference from the monolithic text is due to omitted separators between the separately stored graphs.

### 4.3 Why result ordering matters

A wrong interpretation of a bulk response can produce plausible-looking yet entirely incorrect industrial outputs. In the smaller 512-term, four-graph test, interpreting the result array as graph-major produced a maximum absolute error of approximately **$5.6 \times 10^{-16}$**. Incorrectly interpreting the same response as row-major produced an error of **1.706** in the synthetic output units. Therefore a production interface must validate the declared number of graphs and records, total output count, per-result error flags and array orientation before releasing any result downstream.

*Figure 3. A genuinely evaluated 9 × 9 grid from the live MCP 512-term graph (81 values), plotted with $z = 0.5$ and $T = 27^{\circ}$C. Graph values are displayed after rounding to ten decimal places. This visualizes a synthetic response surface and does not represent measured thermal deformation.*

## 5. Experimental protocol and numerical evidence

The model uses a deterministic, platform-independent index-to-coefficient rule and serializes every coefficient with twelve decimal places. Reference values are calculated independently in native JavaScript using the same serialized coefficients, rather than by calling paceval internally. The comparison checks absolute differences $|\delta_{\text{MCP}} - \delta_{\text{JS}}|$, result count, error flags and the identities of the value sets. Different addition orders can yield slightly different floating-point results; **bitwise identical results were not required**.

| Tested graph design | Formula length | Input records | Server-reported calculation time | Largest observed absolute discrepancy |
|---|---|---|---|---|
| One graph, 64 terms | 9,238 chars | 12 | 0.003207 s | $5.6 \cdot 10^{-16}$ |

| Tested graph design | Formula length | Input records | Server-reported calculation time | Largest observed absolute discrepancy |
|---|---|---|---|---|
| One graph, 512 terms | 73,747 chars | 16 | 0.030633 s | $9.4 \cdot 10^{-16}$ |
| Four graphs, 512 terms total | Four × approx. 18,400 chars | 16 | 0.028393 s | $5.6 \cdot 10^{-16}$ |
| One graph, 2,048 terms | 294,928 chars | 12 | 0.056151 s | $1.0 \cdot 10^{-15}$ |
| One graph, 8,192 terms | 1,179,666 chars | 6 | 0.097010 s | $1.6 \cdot 10^{-15}$ |
| One graph, 16,384 terms | **2,359,315 chars** | 12 | **0.524339 s** | $5.3 \cdot 10^{-15}$ |
| Eight graphs, 16,384 terms total | 2,359,308 chars in eight graphs | 12 × 8 | **0.767869 s** | $1.9 \cdot 10^{-15}$ |

The table reports **single experimental observations** of the internal calculation times returned by the service. Different row counts and graph partitions make many rows unsuitable for direct speed comparisons. No full client-side network benchmark, concurrency scaling experiment or CPU/GPU/C++ competitor study was performed here. These quantities should be measured separately in a formal industrial pilot.

*Figure 4. The only strictly comparable pair in the table in terms of mathematical content and input records is the monolithic versus eight-graph 16,384-term computation. These are two single observations and do not establish a general performance ranking.*

Numerical validation has three further limits. First, an independently coded implementation of the **same mathematical formula** does not validate the underlying physics. Second, absolute error near $10^{-15}$ for selected synthetic records does not establish uniform accuracy over the full four-dimensional domain. Third, the intervals provided by paceval's optional TINC facility were **not** used or validated in these MCP experiments.

# 6. Service design, governance and security

A production-oriented service would need to manage model identity independently of a temporary computation handle. A suitable model record might include a human-approved mathematical specification, a content hash, formula variable order, calibrated physical units, machine or asset class, parameter version, creation timestamp, test suite identifier and a permitted-use category. The paceval handle is then a temporary execution reference, **not** the scientific source of truth.

Each production run should record the model version, input dataset version, server/runtime version, result checksum, error state, numerical tolerance and provenance of the calibration data. Graph creation should occur only after formula syntax checks, allowed-operation checks and resource budgeting; untrusted model strings must not be passed directly from unreviewed user input into production decisions.

The public endpoint is suitable only for **synthetic, non-confidential study data** unless separate access control and data-processing agreements explicitly authorize more. A real factory deployment must document protected transport, authentication, authorization, model confidentiality, retention policies, availability, quotas, monitoring, failures and restore procedures. The experiment did not measure those enterprise-service properties.

Because an external mathematical service introduces network variability, this work is restricted to supervisory reporting, replay and engineering analysis. A hard real-time or safety-related controller

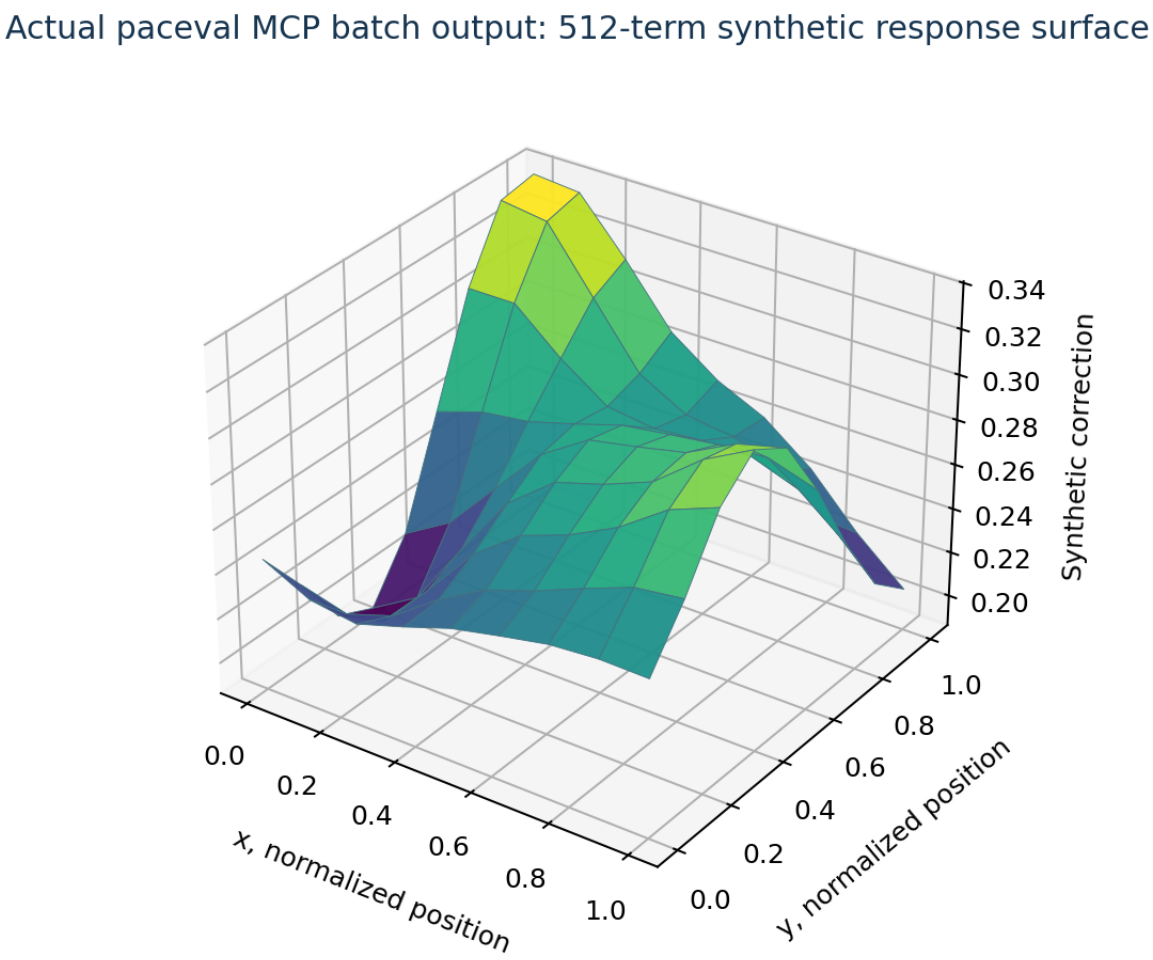



Figure 3: Actual paceval batch-evaluated field for 512 Gaussian terms

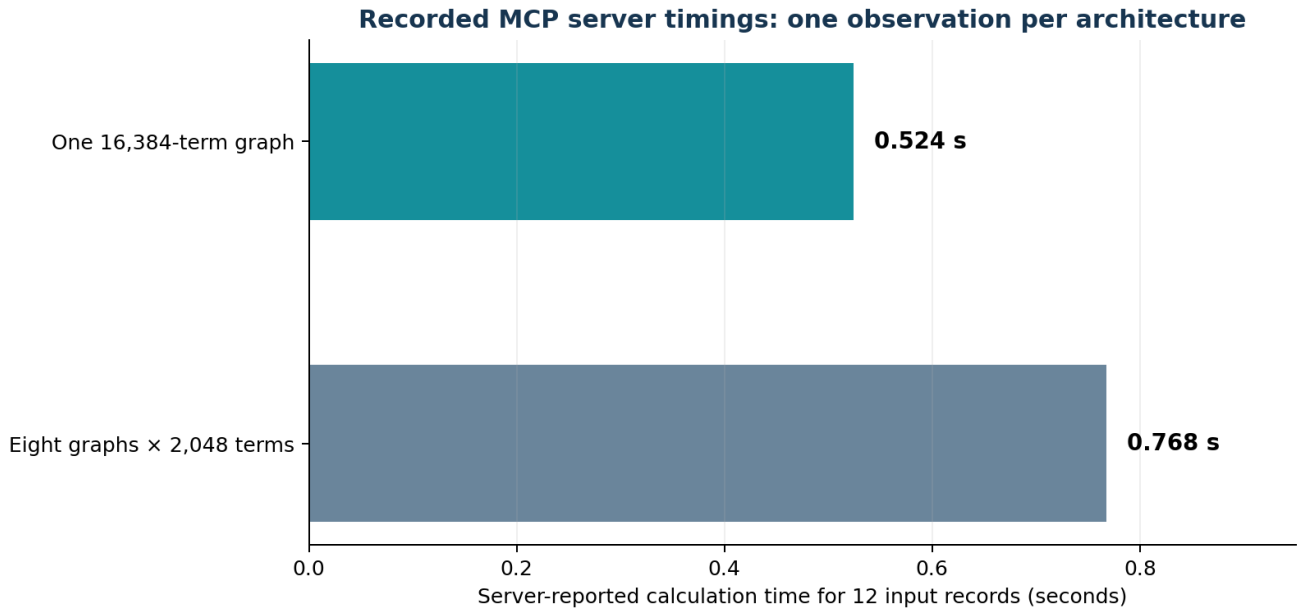



Figure 4: Individual backend timing observations

must not depend on a remote calculation without a separate engineering safety case and local fallback.

# 7. Industrial research program: five work packages

**Work package A – Calibration and data selection.** Obtain actual measured position errors and temperatures from an approved research partner. Specify units, sensor uncertainty, operating histories, thermal transients and machine identities. Reserve entire time windows or machines for out-of-sample testing to avoid data leakage. Compare simple linear, polynomial and lookup-table baselines with the proposed analytical basis-function model. The target is to identify a useful mathematical model, **not** to maximize its text length.

**Work package B – Analytical representation and update workflow.** Serialize validated models into persistent mathematical expressions. Measure creation success, graph size, update frequency, reproducibility across model revisions and lifecycle of computation handles. Register both monolithic and modular forms. Define deterministic hashes so identical text can be recognized and audited across engineering teams.

**Work package C – Numerical and service validation.** Compare outputs against an independent high-precision reference on random, extreme and ill-conditioned inputs. Record absolute, relative and unit-in-last-place errors where appropriate. Characterize server errors, dropped connections, expired handles, duplicate requests and recovery after service restarts. No claims concerning certified numerical enclosures may be made without an independent interval study.

**Work package D – Fair architectural comparison.** Use identical measurement data, mathematical definitions and numerical tolerances for paceval MCP and established alternatives. Measure server graph creation separately from cache reuse, batch evaluation, network transfer, JSON processing and the complete engineering workflow. Report p50, p95 and p99 latency, throughput, CPU and memory demand, service availability and cost per accepted analysis. Allow optimized C++ implementations and established interpolation packages to compete on equal terms.

**Work package E – Operational value.** Conduct controlled model-maintenance exercises, such

as ten approved changes to a calibration function, a requalification after a sensor replacement, and an audit replay at two sites. Measure engineering hours, error incidence, deployment delay and traceability. A possible business case is better reuse and governance even if the numerical kernel is not the fastest method.

The research should be discontinued for an industrial application if it cannot satisfy accuracy and confidentiality requirements, or if a simpler validated reference approach offers lower total cost with equal maintainability. These are scientific decision criteria, not predictions of failure.

# 8. Reproducibility of the present demonstration

This package contains the exact generated formulas (`generated/`), SHA-256 manifest, the deterministic JavaScript model generator, an independent JavaScript reference implementation, an optional Node.js MCP SDK client, machine-readable result summaries and the 81-point genuine MCP plot data. The executable **connected-chat MCP results** were obtained through `paceval_create_computation`, `paceval_evaluate_many`, `paceval_evaluate_multiple_many` and `paceval_get_error_information`; they require no Python scripts.

To regenerate the exact mathematical texts locally:

```
node src/generate_formulas.mjs
node tests/offline_tests.mjs
```

The largest regenerated file must have **2,359,315 ASCII characters** and SHA-256 `7fffe2b0b7bab11b63a4026891b5`
These instructions generate and test formulas and references locally; they do **not** contact the MCP server.

An optional standalone MCP client built with the official TypeScript/JavaScript SDK is provided in `src/mcp_client.mjs`. It requires a reachable endpoint and any authentication required by that endpoint. The code is clearly marked **source-checked but not network-executed here**; the positive live results in this study instead come from the installed ChatGPT MCP integration. This avoids conflating an untested standalone transport with an actually executed service call. [6]

For example, a standalone test after installing its SDK dependency may be started with:

```
npm install
node src/mcp_client.mjs --terms 512 --rows 12 --chunks 4 --mode both --execute
```

An agent with the paceval MCP connector can run the identical sequence by generating a formula, calling `paceval_create_computation`, preserving the returned handle and passing it to the batch tool. The original opaque handles are not permanent database identifiers; successful reuse was observed during this session, but their later availability cannot be assumed.

# 9. MCP connection and independent research handoff

## 9.1 Server address and prerequisite

The public paceval MCP endpoint described in the paceval announcement is `https://mcp.paceval-service.com/mcp`
It is a remote MCP service, **not** the Python REST API, the local Docker service on port 8080, or a web browser URL for interactive page viewing. A compatible MCP client and permitted network access are necessary. Connection permissions and any authentication requirements must be checked in the chosen client; a public URL alone does not establish a working connection. The observed paceval runtime in this research session reported version **4.26**. [5]

## 9.2 ChatGPT setup (where custom MCP connections are enabled)

The current ChatGPT web interface supports a straightforward setup for eligible **Business and Enterprise/Edu** workspaces, subject to administrator and workspace permissions. It should **not**

be represented as universally available to every ChatGPT account. The verified steps in the current Plugins interface are:

1. Open **Plugins**, select **+**, then **Add custom MCP server**.
2. Name the connection **paceval MCP Server** and enter endpoint `https://mcp.paceval-service.com/mcp`.
3. Select the server's required authentication mode if requested; do not invent credentials or tokens. Review the server and acknowledge the connection risk warning.
4. Choose **Create as a plugin** and inspect the discovered tool names. Workspace admins may need to grant access or publish the plugin before other members can install or use it.
5. Start a new conversation, select the installed plugin (for example by typing **@** and selecting it), and run a small test expression before attempting long formulas.

OpenAI's current documentation says that no separate Developer Mode switch is required in this Plugins setup. However, access depends on plan, administrative permissions and supported ChatGPT surfaces. See OpenAI: Connecting custom MCP servers in ChatGPT. [8]

## 9.3 Other MCP-capable LLM clients

Configure a **remote HTTP MCP server** or **Streamable HTTP MCP** connection, as the client's current documentation permits, with the same endpoint URL. If the client requires a server type, choose the remote HTTP transport; do not substitute an arbitrary local stdio command. Authorization, TLS trust and available tools must be validated for that particular client. Tool naming and connection menus may differ by product. A client that cannot reach external MCP services cannot reproduce the live experiment merely from this paper.

## 9.4 Expected operations and a required smoke test

The connected service in this research exposed `paceval_calculate`, `paceval_create_computation`, `paceval_evaluate`, `paceval_evaluate_many`, `paceval_evaluate_multiple`, `paceval_evaluate_multiple_many` and `paceval_get_error_information`. Tool exposure is client- and server-specific; re-discover the list on the new connection. The essential reusable workflow is:

1. **Create:** `paceval_create_computation` with expression `x*x+sin(y)`, variables `['x','y']`, interval disabled.
2. **Reuse the exact handle:** `paceval_evaluate_many` with value sets `[[2,0],[3,0]]`. Expected mathematical results are **4** and **9**.
3. **Extend:** register longer formulas once, preserve each exact opaque handle, and evaluate numerous value sets via one batch operation.
4. **Compare architectures:** register equal-variable-order subgraphs, evaluate them with `paceval_evaluate_multiple_many`, verify returned shapes and error numbers, and only then sum graph-major outputs.

Computation handles are service-owned, potentially expiring resources. The literal handles recorded in previous experiment logs are **not** reusable credentials and must not be pasted into a fresh session as if they were guaranteed valid.

## 9.5 Reproducibility package and prompt for another ChatGPT session

The archive contains `docs/RESEARCH_HANDOFF_PROMPT_EN.md`, a self-contained English prompt for a new MCP-equipped AI conversation. Supply the **report and complete research package** as files, install/enable the paceval MCP connection in that new session, and then paste the prompt. The receiving agent should rerun the small smoke test before the larger 2.36-million-character model and distinguish all new measurements from historical figures. Formula generation and independent reference checks can use the supplied JavaScript scripts; actual paceval evaluation must go through MCP.

# 10. Scientific conclusions and next decision

The new empirical contribution is clear: the connected paceval MCP service successfully created and evaluated a **multi-million-character analytical expression** as a reusable graph, and separately processed the same mathematical model as a **multi-graph batch with 96 outputs**. Independent JavaScript calculations support the numerical consistency of those selected results. This resolves the main weakness of the earlier local Python/HTTP prototype: here a configured **real MCP service** was used.

It does **not** establish that the method is faster, more accurate for measured machines, cheaper in production, or superior to existing mathematical libraries. A complex formula is a means to represent a validated model, not a proxy for industrial value. The most credible next step is a small, realistic pilot with genuine calibration measurements and a controlled model-update workflow. A positive study would demonstrate reduced model management effort, consistent execution across engineering tools and an acceptable complete service cost at independent accuracy targets.

## References and source documentation


[1] *Digital Twin for Volumetric Thermal Error Compensation of Large Machine Tools.* Sensors (2024), DOI: 10.3390/s24196196.
[2] S. Lang et al., *Sensor placement utilizing a digital twin for thermal error compensation of machine tools.* Journal of Manufacturing Systems 80 (2025), DOI: 10.1016/j.jmsy.2025.03.003.
[3] X. Fu et al., *Digital twin technology in modern machining: A comprehensive review of research on machining errors.* Journal of Manufacturing Systems 79 (2025), DOI: 10.1016/j.jmsy.2025.01.005.
[4] G. Shao, J. Hightower and W. Schindel, *Credibility consideration for digital twins in manufacturing.* Manufacturing Letters (2022), NIST publication.
[5] paceval, *paceval now available as an MCP Server for AI Agents and LLMs*, August 2026, official announcement.
[6] Model Context Protocol TypeScript SDK, *Connect to an MCP server*, official documentation.
[7] paceval, *Mathematical Engine and public API*, paceval API.
[8] OpenAI Help Center, *Connecting custom MCP servers in ChatGPT*, current instructions.